

Aula 18: Modelagem Generativa e Normalizing Flows

Coupling Layers e Implementação em 2D

Disciplina de Aprendizagem Profunda

Objetivos da Aula

- Entender a diferença entre **modelos discriminativos** e **modelos generativos**.
- Ver princípios gerais de modelagem generativa:
 - amostragem a partir de distribuições simples;
 - transformação de amostras;
 - composição de transformações.
- Introduzir **Normalizing Flows** e a fórmula de mudança de variáveis.
- Estudar **coupling flows** (RealNVP/Glow) em 2D.
- Implementar um flow simples e ajustá-lo ao dataset TwoSpirals.

Discriminativos vs Generativos I

Modelos discriminativos

Aprendem diretamente:

$$p_{\theta}(y \mid x)$$

- Ex.: regressão logística, MLPs de classificação, ResNets, BERT para classificação.
- Foco: fronteira de decisão, acurácia de predição.

Discriminativos vs Generativos II

Modelos generativos

Aprendem:

$$p_{\theta}(x) \quad \text{ou} \quad p_{\theta}(x, y)$$

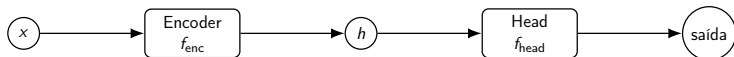
- Gerar novas amostras x plausíveis.
- Completar dados, fazer *inpainting*, simular mundos possíveis.
- Permitem inferir $p_{\theta}(y \mid x)$ se tivermos o conjunto.

Aprendizagem de Representações em Modelos Profundos

- Em quase todo modelo profundo, podemos separar:

$$h = f_{\text{enc}}(x; \theta_{\text{enc}}), \quad \text{saída} = f_{\text{head}}(h; \theta_{\text{head}}).$$

- f_{enc} : extrai uma **representação** (embedding) de x .
- f_{head} : implementa a tarefa específica.



- A **mesma representação** h pode ser usada em: classificação, regressão, geração, etc.

Três Regimes: Supervisado, Auto-Supervisionado e Generativo

	Supervisionado	Auto-supervisionado	Generativo
Objetivo	Predizer y a partir de x	Criar rótulos a partir de x (máscaras, partes, contrastes)	Modelar $p_\theta(x)$ ou $p_\theta(x, y)$
Loss típica	$\mathbb{E}[\ell(f(x), y)]$	$\mathbb{E}[\ell(f(x), \tilde{y}(x))]$	$-\mathbb{E}[\log p_\theta(x)]$
Encoder	Aprende $h = f_{\text{enc}}(x)$	Aprende $h = f_{\text{enc}}(x)$	Aprende h implícito (latentes, features internas)
Uso típico	Classificação, regressão	Pré-treinamento de representações	Amostragem, simulação, geração de dados

- **Em todos os casos**, existe uma parte do modelo que aprende uma representação interna de x .

Modelagem Generativa como Aprendizagem de Representações

- Em muitos modelos generativos, existe um **latente** z :

$$z \sim p_Z, \quad x \sim p_\theta(x | z).$$

- Podemos ver z como uma **representação comprimida** de x .
- Em flows:

$$z = f_\theta^{-1}(x), \quad x = f_\theta(z).$$

z é uma coordenada em um espaço onde a distribuição é simples.

- Em VAEs:

$$z \sim q_\phi(z | x), \quad x \sim p_\theta(x | z).$$

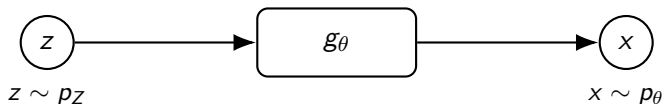
- Assim, modelagem generativa também pode ser vista como:
“aprender boas representações que sejam fáceis de amostrar”.

O que é “gerar dados”? I

- Ideia central: obter um **procedimento de amostragem**

$$z \sim p_Z \quad \Rightarrow \quad x = g_\theta(z)$$

- p_Z é uma distribuição **simples**: Normal padrão, uniforme, etc.
- g_θ transforma ruído em estrutura.



- Em muitos modelos (GANs, score-based), só temos esse gerador implícito.
- Em outros (flows), também queremos a **densidade** $p_\theta(x)$.

Princípios Gerais de Modelagem Generativa I

1 Parametrizar uma distribuição

- explícita: $p_\theta(x)$ disponível (flows, alguns VAEs);
- implícita: só amostrador (GANs, score-based na prática).

2 Amostrar de uma base simples $z \sim p_Z$.

3 Transformar amostras $x = f_\theta(z)$.

4 Compor transformações:

$$x = f_K \circ \cdots \circ f_1(z).$$

5 Controlar a densidade resultante para treinar com MLE.

Composição de Mapas = Profundidade

- Em redes neurais, cada camada é uma função simples.
- A composição gera representações ricas:

$$h^{(L)} = f_L \circ f_{L-1} \circ \cdots \circ f_1(x).$$

- Em modelagem generativa:

$$z \sim p_Z, \quad x = f_K \circ \cdots \circ f_1(z).$$

- A distribuição de x é o **pushforward** de p_Z por f :

$$p_X = f_{\#} p_Z.$$

- Normalizing Flows: escolher f invertível e com Jacobiana controlável.

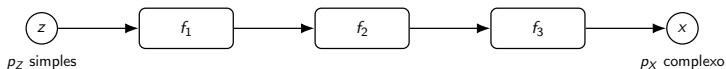
Normalizing Flows: Intuição

- Queremos:

- amostrar facilmente: $z \sim p_Z$, $x = f_\theta(z)$;
- calcular densidades: $p_\theta(x) = ?$.

- Estratégia:

- escolher f_θ **invertível**;
- com Jacobiana de determinante eficiente;
- e compor várias camadas.



Fórmula de Mudança de Variáveis (1D)

Caso unidimensional: $x = f(z)$, f invertível e diferenciável.

$$p_X(x) = p_Z(z) \left| \frac{dz}{dx} \right|.$$

Como $z = f^{-1}(x)$, temos

$$p_X(x) = p_Z(f^{-1}(x)) \left| \frac{d}{dx} f^{-1}(x) \right|.$$

Intuição:

- conservamos probabilidade:

$$p_Z(z)dz = p_X(x)dx.$$

- logo, $p_X(x) = p_Z(z) \frac{dz}{dx}$.

Mudança de Variáveis em $\mathbb{R}^D$

Agora, $x = f(z)$, com $z, x \in \mathbb{R}^D$, f bijetora e diferenciável.

$$p_X(x) = p_Z(f^{-1}(x)) \left| \det \left(\frac{\partial f^{-1}(x)}{\partial x} \right) \right|.$$

Usando $z = f^{-1}(x)$, também podemos escrever:

$$p_X(x) = p_Z(z) |\det J_f(z)|^{-1},$$

onde $J_f(z)$ é a Jacobiana de f em z .

Em flows:

- direção *forward* para amostragem: $z \rightarrow x$;
- direção *inverse* para calcular log-likelihood: $x \rightarrow z$.

Composição de Flows

Seja

$$x = f_K \circ f_{K-1} \circ \cdots \circ f_1(z).$$

Então:

$$\log p_X(x) = \log p_Z(z_K) + \sum_{k=1}^K \log |\det J_{f_k}(z_{k-1})|^{-1},$$

com $z_0 = z$, $z_k = f_k(z_{k-1})$.

Na prática:

- empilhamos bijeções simples;
- cada uma tem Jacobiana de determinante fácil.

Exemplo Simples: Affine Flow em 1D

$$\begin{aligned} z &\sim \mathcal{N}(0, 1), \\ x &= az + b, \quad a \neq 0. \end{aligned}$$

Então:

$$z = \frac{x - b}{a}, \quad \frac{dz}{dx} = \frac{1}{a}.$$

Logo,

$$p_X(x) = p_Z\left(\frac{x - b}{a}\right) \left|\frac{1}{a}\right|.$$

- Isso é um flow de uma camada.
- Em alta dimensão, precisamos de camadas mais espertas.

Coupling Layers (RealNVP/Glow): Ideia

- Dividimos o vetor x em duas partes: $x = (x_A, x_B)$.
- Mantemos x_A fixo, e transformamos x_B usando x_A :

$$(s, t) = \text{NN}(x_A),$$

$$z_A = x_A, \quad z_B = (x_B - t) \odot e^{-s}.$$

- Jacobiana triangular:

$$\log |\det J| = - \sum s.$$

Coupling Layer 2D: Fórmulas I

Para o caso 2D, $x = (x_1, x_2)$:

Forward (dados $\rightarrow$ base):

$$(m, \alpha) = \text{NN}(x_1),$$

$$z_1 = x_1,$$

$$z_2 = (x_2 - m) \exp(-\alpha),$$

$$\log |\det J| = -\alpha.$$

Coupling Layer 2D: Fórmulas II

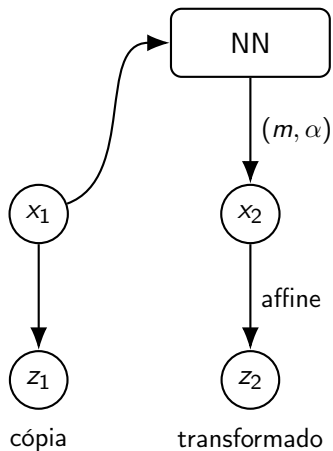
Inverse (base $\rightarrow$ dados):

$$(m, \alpha) = \text{NN}(z_1),$$

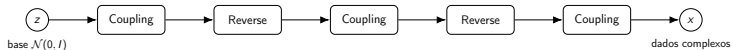
$$x_1 = z_1,$$

$$x_2 = z_2 \exp(\alpha) + m.$$

Diagrama: Coupling Layer em 2D



Arquitetura: Pilha de Coupling + Reverse



Dataset: TwoSpirals

- Dataset sintético 2D, duas espirais intercaladas.
- Bom para testar a capacidade de um modelo em aprender distribuições multimodais não lineares.

Modelo Flow em Código (Visão Geral) I

Classe Flow

- guarda uma lista de bijeções: `ModuleList(bijections);`
- `base_dist`: $\mathcal{N}(0, I)$ em 2D;
- `log_prob(x)`:
 - aplica cada bijeção `(x, ldj) = bijection(x);`
 - acumula `ldj`;
 - soma com `base_dist.log_prob(x)`.
- `sample(num_samples)`:
 - amostra z da base;
 - aplica bijeções na ordem inversa usando `inverse`.

Reverse e Coupling: Código Lógico

Reverse

- inverte a ordem das dimensões: $[x_1, x_2] \mapsto [x_2, x_1]$;
- Jacobiana é uma permutação, determinante $\pm 1 \Rightarrow \log\text{-det}$ zero.

Coupling

- divide x em duas partes;
- usa a primeira parte como condição para transformar a segunda;
- produz saída e ldj .

Objetivo de Treino

- Queremos maximizar a log-verossimilhança:

$$\max_{\theta} \mathbb{E}_{x \sim \text{dados}} [\log p_{\theta}(x)].$$

- Na prática, minimizamos a perda:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log p_{\theta}(x_i).$$

- Usaremos Adam com `lr=1e-3`, minibatches e `train_loader`.

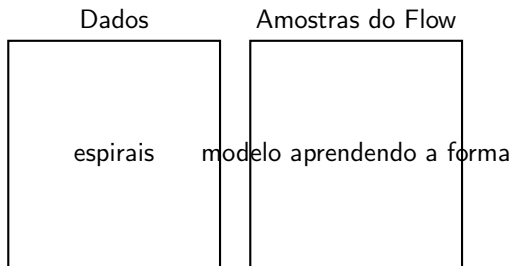
Loop de Treino (Pseudo-código)

Treino

- for epoch in range(epochs):
 - for x in train_loader:
 - enviar x para device;
 - `loss = - flow.log_prob(x).mean();`
 - `loss.backward();`
 - `optimizer.step();`
- monitorar `loss_sum/len(train_loader)`.

Visualização: Antes e Depois

- No início: `flow.sample(...)` gera algo próximo de Normal 2D.
- Depois do treino: as amostras devem aproximar as duas espirais.
- Usamos `hist2d` para visualizar densidades.



Experimentos Extras

- Variar:
 - número de coupling layers;
 - dimensão da MLP interna (`hidden_dim`);
 - número de camadas da MLP;
 - função de ativação (ReLU, Tanh, LeakyReLU).
- Observar:
 - qualidade visual das amostras;
 - estabilidade e velocidade do treino;
 - overfitting vs. underfitting.

Resumo da Aula

- Revisamos modelagem generativa vs discriminativa.
- Vimos princípios gerais: amostragem, transformações, composição.
- Derivamos a fórmula de mudança de variáveis em 1D e em D dimensões.
- Introduzimos Normalizing Flows e coupling layers.
- Implementamos (no notebook) um flow simples para o dataset TwoSpirals.

Dúvidas?